

15+5 Minutos

Template Method

The Sacred Elements of the Faith

the holy
origins

the holy
structures

the holy
behaviors

107 FM Factory Method							139 A Adapter
117 PT Prototype	127 S Singleton				223 CR Chain of Responsibility	163 CP Composite	175 D Decorator
87 AF Abstract Factory	325 TM Template Method	233 CD Command	273 MD Mediator	293 O Observer	243 IN Interpreter	207 PX Proxy	185 FA Façade
97 BU Builder	315 SR Strategy	283 MM Memento	305 ST State	257 IT Iterator	331 V Visitor	195 FL Flyweight	151 BR Bridge

Patrones de Comportamiento

Foco

- **Cómo los objetos interactúan y se comunican entre sí,**
- **Cómo distribuir responsabilidades de manera flexible.**

Objetivo:

- **Delegar responsabilidades** entre objetos.
- **Controlar el flujo** de información.
- Aumentar la **flexibilidad en la comunicación** sin acoplar demasiado las clases.

⇒ Diseñar la **dinámica del sistema**, más que su composición (*estructural*) o su creación (*creacional*).

The Sacred Elements of the Faith

the holy
origins

the holy
structures

107 FM Factory Method							139 A Adapter
117 PT Prototype	127 S Singleton				223 CR Chain of Responsibility	163 CP Composite	175 D Decorator
87 AF Abstract Factory	325 TM Template Method	233 CD Command	273 MD Mediator	293 O Observer	243 IN Interpreter	207 PX Proxy	185 FA Façade
97 BU Builder	315 SR Strategy	283 MM Memento	305 ST State	257 IT Iterator	331 V Visitor	195 FL Flyweight	151 BR Bridge

Coffee time!

1. Tostar Cafe
2. Calentar el agua
3. Mezclar Agua y Cafe
4. Filtrar la Mezcla
5. Verter en “vaso”
6. Beber



beber(vertir(vaso, filtrar(mezclar(calentar(agua), tostar(cafe))

Transferencia Fondos

transferMoney(monto)

validateSourceAccount()

checkSufficientBalance()

deductFromSource()

executeTransfer()

logTransaction()

notifyUser()

checkSufficientBalance()

Descubierto > saldo - monto?

Prestamo > saldo - monto?

executeTransfer()

Misma Plataforma: Inmediata, \$0

HCI: Mismo Día, 2% <=\$2.5

Wire: 24hs, \$2 + 0.2%

Template Method

- **Propósito:**

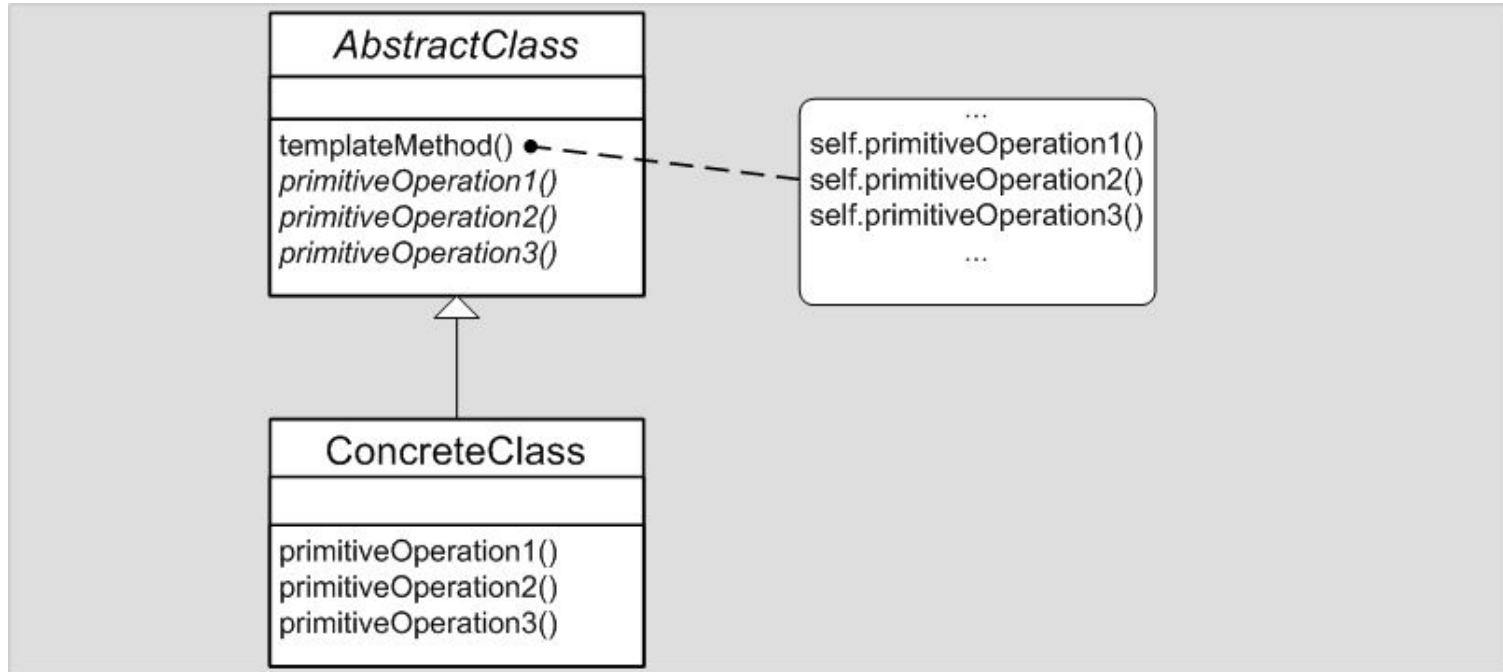
Definir el esqueleto de un algoritmo en un método, difiriendo algunos pasos a las subclases.

Template Method permite que las subclases redefinan ciertos pasos de un algoritmo sin cambiar la estructura del algoritmo.

- **Aplicabilidad:** usar Template Method

- para implementar las partes invariantes de un algoritmo una vez y dejar que las subclases implementen los aspectos que varían;
- para evitar duplicación de código entre subclases;
- para controlar las extensiones que pueden hacer las subclases

Template Method. Estructura



Template Method.Participants

✓ **AbstractClass**

- Implementa un método que contiene el esqueleto de un algoritmo (el template method). Ese método llama a operaciones primitivas así como operaciones definidas en AbstractClass
- Declara operaciones primitivas abstractas que las subclases concretas deben definir para implementar los pasos de un algoritmo

✓ **ConcreteClass**

- Implementa operaciones primitivas que llevan a cabo los pasos específicos del algoritmo

Template Method

● Consecuencias

- Técnica fundamental de reuso de código
- Lleva a tener **inversión de control** (la superclase llama a las operaciones definidas en las subclases)
- El template method llama a dos tipos de operaciones:
 - operaciones primitivas (abstractas en AbstractClass y que las subclases tienen que definir)
 - operaciones concretas definidas en AbstractClass y que las subclases pueden redefinir si hace falta (hook methods)

● Implementación:

- minimizar la cantidad de operaciones primitivas que las subclases deben redefinir
-

Template Method

Consecuencias

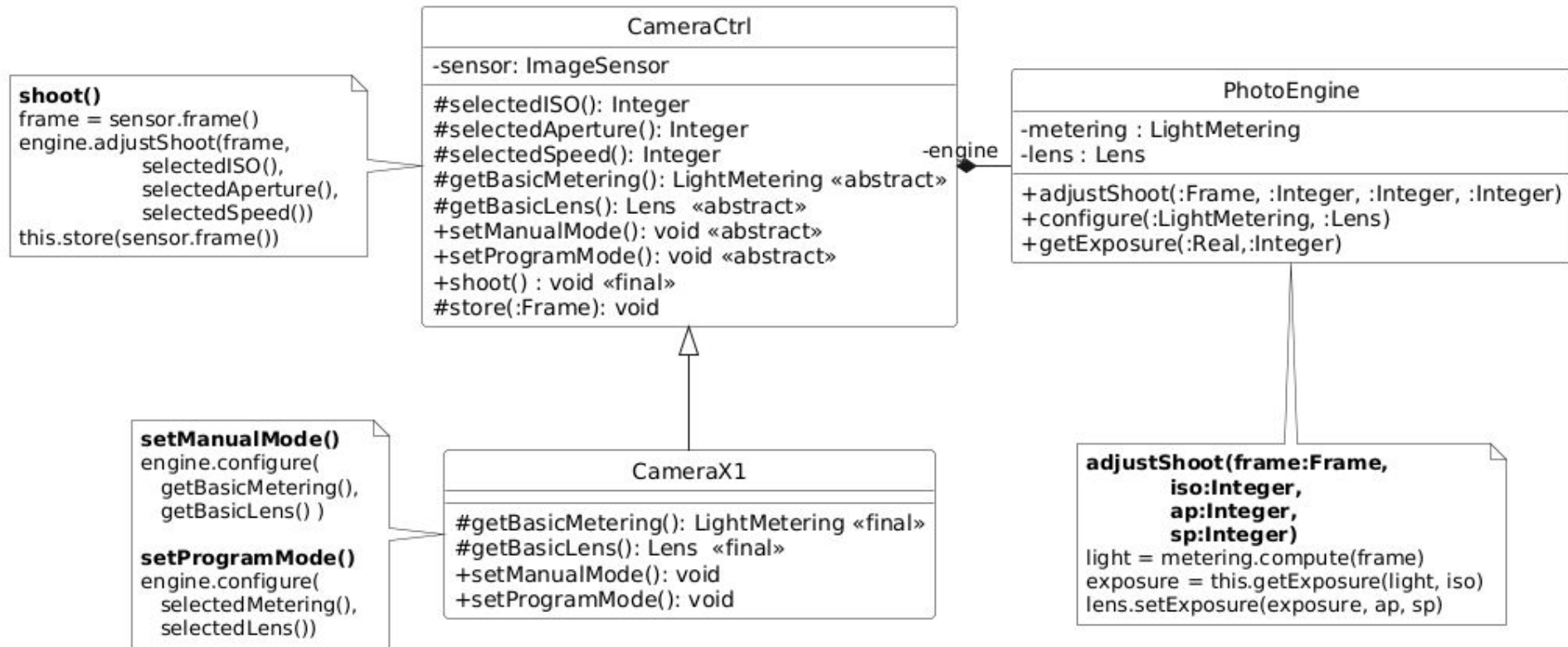
- Técnica fundamental de reuso de código
- Lleva a tener inversión de control (la superclase llama a las operaciones definidas en las subclases)
- El template method llama a dos tipos de operaciones:
 - operaciones primitivas (abstractas en AbstractClass y que las subclases tienen que definir)
 - operaciones concretas definidas en AbstractClass y que las subclases pueden redefinir si hace falta (hook methods)

Implementación:

- Operaciones Primitivas sólo deben ser accesibles por el template method (protected)
- Minimizar la cantidad de operaciones primitivas que las subclases deben redefinir

Template Method. Resumen

- Es un patrón de diseño de “tiempo de compilación”
 - Herencia
 - Polimorfismo
- Ideas contrastantes
 - Definición única de un algoritmo basado en pasos (análisis de dominio)
 - Existen variantes que resultan en múltiples comportamientos/funcionalidades heterogéneas
- Subclases pueden implementar primitivas vacías
 - Ej: transferencias con \$0 de costo
 - Ej: mecanismos que ante un paso dado no hacen nada
- No-nos
 - No existe una definición única, no se distinguen pasos
 - Subclases que reimplementan el template
 - Subclases que invocan directamente (super) métodos de la superclase



CameraCtrl::shoot() es un template method? Justifique

CameraX1::setManualMode() es un template method? Justifique